

# 第一章：软件问题（The Software Problem）

参考教材：Pankaj Jalote, 《软件工程导论》(A Concise Introduction to Software Engineering)  
本章重点：理解软件开发的复杂性、成本、进度与质量之间的平衡，以及规模与变化对软件的影响。

## 学习目标

学习本章后，你应能够：

- 了解软件开发中常见的核心问题
- 理解成本、进度、质量三者的相互制约关系
- 掌握“规模扩大”和“需求变化”对软件项目的影响
- 理解为什么需要软件工程方法

## 1.1 成本、进度和质量（Cost, Schedule, and Quality）

### 软件项目的三角关系

软件工程中有一个著名的“铁三角”模型：

**成本（Cost） ↔ 进度（Schedule） ↔ 质量（Quality）**

这三者互相制约：

- 若想**提前完成（进度）**，通常需增加成本或牺牲质量；
- 若想**提升质量**，往往要花更多时间和资金；
- 若想**降低成本**，可能会影响进度与质量。

| 目标   | 可能影响      |
|------|-----------|
| 缩短进度 | 成本上升，质量下降 |
| 提高质量 | 成本上升，时间增加 |
| 降低成本 | 进度延长，质量下降 |

### ■ 例子：

某公司要求三个月内完成原计划六个月的系统开发 →  
结果开发团队加班赶工，最终系统虽交付，但缺陷率高、维护成本增加。

## 软件质量的维度

软件质量不仅仅是“能运行”，还包括：

- 功能性 (Functionality)
- 可靠性 (Reliability)
- 可维护性 (Maintainability)
- 可移植性 (Portability)
- 可用性 (Usability)

### 工程思想：

在有限成本和时间内，通过科学方法最大化软件质量。

## 1.2 规模和变更 (Scale and Change)

### 软件规模问题

随着系统规模增大：

- 模块数量、接口复杂度、人员协作成本急剧上升；
- 小型项目的经验往往无法直接套用到大型项目；
- 沟通、测试、集成成为主要挑战。

### ■ 举例：

一个10人月项目可能只需基本文档；  
但一个1000人月项目必须有完整的需求、设计、测试、配置管理体系。

### 软件变更问题

“软件唯一不变的，就是变化本身。”

- 需求会随时间变化（客户新想法、环境变化、技术更新）；
- 代码、设计、接口都需适应这些变化；
- 维护阶段的变更成本往往高于开发阶段数倍。

变更的主要来源

| 来源   | 示例        |
|------|-----------|
| 需求变更 | 用户提出新功能   |
| 技术变更 | 新平台、新语言   |
| 法规变更 | 隐私政策更新    |
| 环境变更 | 操作系统或硬件更新 |

变更的应对策略

- 1. **模块化设计 (Modularity)**：降低耦合度，便于局部修改。
- 2. **版本控制 (Version Control)**：跟踪代码变化 (Git、SVN)。
- 3. **文档化 (Documentation)**：明确设计意图。
- 4. **持续测试与维护 (Continuous Testing & Maintenance)**。

结论：

软件工程的核心目标之一，就是系统性地应对变化和复杂性。

1.3 小结 (Summary)

| 主题       | 关键内容           |
|----------|----------------|
| 成本、进度、质量 | 三者互相制约，需权衡     |
| 规模问题     | 系统越大，复杂度越高     |
| 变化问题     | 需求与环境变化不可避免    |
| 软件工程     | 提供系统化方法以应对这些挑战 |

✔ 软件问题的根源在于“复杂性 + 变化”，  
软件工程的使命就是“控制复杂性 + 管理变化”。



# 自测练习 (Self-Review)

## 一、选择题

1. 软件项目中，成本、进度和质量的关系是（）。  
A. 可以同时最大化 B. 相互独立 C. 彼此制约 D. 无关联
2. 当软件规模扩大时，以下哪项**不会**显著增加？（）  
A. 模块接口 B. 团队沟通 C. 代码行数 D. 逻辑清晰度
3. 需求变化的常见来源不包括（）。  
A. 用户需求 B. 技术发展 C. 程序调试 D. 法律法规

## 二、判断题

1. ( ) 软件规模增大后，复杂性仍能保持不变。
2. ( ) 提高软件质量可能导致成本和进度上升。
3. ( ) 软件工程的一个核心任务是应对变化。

## 三、简答题

1. 简述软件工程中“成本、进度、质量”的三角关系。
2. 为什么说“软件维护阶段的工作量可能超过开发阶段”？
3. 请举例说明软件变化的几种类型。



## 答案参考

### 选择题：

1. C 2. D 3. C

### 判断题：

1. × 2. √ 3. √

### 简答题要点：

1. 三者相互制约，改变一个通常会影响另两个。
2. 因为软件在使用后需长期修复缺陷、适应环境变化、添加功能。
3. 示例：客户新增功能（需求变化）、新法规出台（外部变化）、新技术框架采用（技术变化）。

## 第二章：软件过程（Software Process）

教材参考：Pankaj Jalote, 《软件工程导论》(A Concise Introduction to Software Engineering)

本章目标：理解软件开发的整体过程、主要阶段及典型模型，为项目规划与执行奠定基础。

### 学习目标（Learning Objectives）

学习完本章后，你应该能够：

- 理解软件过程与项目的关系
- 掌握软件过程的主要组成部分
- 熟悉常见的软件开发过程模型（如瀑布、原型、迭代、敏捷等）
- 理解 Rational 统一过程（RUP）的核心思想
- 掌握敏捷与极限编程（XP）的原则
- 理解过程模型在项目中的实际应用

## 2.1 过程和项目（Process and Project）

定义：

- **软件过程（Software Process）：**  
指一组相互关联的活动，用于将需求转化为软件产品。
- **软件项目（Software Project）：**  
是一次性实施的软件开发任务，通常由团队在有限时间与资源内完成。

关系：

- 软件过程定义了“如何做”；
- 软件项目是在具体环境中“实际去做”。

 示意图：

- 软件过程（通用框架）
- ↓
- 软件项目（具体实施）

## 2.2 软件过程的组成 (Components of a Software Process)

一个完整的软件过程包括以下主要活动：

### 1. 需求分析 (Requirements Analysis)

理解并记录客户需求。

### 2. 设计 (Design)

制定系统架构与模块设计方案。

### 3. 实现 (Implementation / Coding)

将设计转化为可执行代码。

### 4. 测试 (Testing)

验证系统是否满足预期需求。

### 5. 维护 (Maintenance)

修复缺陷与持续改进。

## 2.3 软件开发过程模型 (Software Development Process Models)

### 2.3.1 瀑布模型 (Waterfall Model)

特点：

- 线性、顺序执行，每个阶段完成后才能进入下一个阶段。
- 强调文档化与阶段性评审。

优点：

- 结构清晰、易于管理
- 适用于需求稳定的项目

缺点：

- 缺乏灵活性
- 需求变更代价高

■ 典型流程：

- 需求 → 设计 → 编码 → 测试 → 部署 → 维护

## 2.3.2 原型模型（Prototyping Model）

**核心思想：**

通过快速构建原型，获取用户反馈，再逐步完善系统。

**适用场景：**

- 需求不明确
- 用户参与度高的项目

**优点：**

- 改善需求理解
- 提升用户满意度

## 2.3.3 迭代开发模型（Iterative Model）

**思想：**

将软件分成多个小版本，每次循环包括需求、设计、实现、测试。

**优点：**

- 提供早期可运行版本
- 降低项目风险
- 便于持续改进

 **循环结构示意：**

- 需求 → 设计 → 编码 → 测试 → 评估 → 下一次迭代

## 2.3.4 Rational 统一过程模型（Rational Unified Process, RUP）

**核心思想：**

- 基于迭代开发
- 强调用例驱动（Use Case Driven）
- 注重架构设计（Architecture Centric）

**四个阶段：**

| 阶段              | 目标      |
|-----------------|---------|
| 启动阶段（Inception） | 明确范围与目标 |

| 阶段                 | 目标     |
|--------------------|--------|
| 细化阶段（Elaboration）  | 建立体系结构 |
| 构建阶段（Construction） | 完成主要功能 |
| 移交阶段（Transition）   | 交付与部署  |

### 2.3.5 时间盒模型（Time Box Model）

特点：

- 为每次开发迭代设定固定时间限制（时间盒）
- 在限定时间内交付可运行版本

优点：

- 控制项目节奏
- 强调优先级与交付结果

### 2.3.6 极限编程（XP）和敏捷过程模型（Agile & Extreme Programming）

核心思想：

- 以人为本，强调沟通与反馈
- 短迭代（1~2 周），持续交付可运行软件

XP 的主要实践：

1. 测试驱动开发（TDD）
2. 结对编程（Pair Programming）
3. 持续集成（Continuous Integration）
4. 小步前进与重构
5. 客户在场（On-site Customer）

敏捷宣言（Agile Manifesto）核心价值：

| 更重视    | 而不是   |
|--------|-------|
| 个体与互动  | 流程与工具 |
| 可工作的软件 | 详尽的文档 |



| 更重视  | 而不是  |
|------|------|
| 客户合作 | 合同谈判 |
| 应对变化 | 遵循计划 |

2.3.7 过程模型在项目中的应用（Applying Models in Projects）

- 大型系统：→ 瀑布模型、RUP
- 创新型项目：→ 原型或迭代模型
- 快速交付与频繁变更：→ 敏捷、XP
- 混合模型：实际项目中常根据情况灵活组合使用

■ 组合示例：

- 需求阶段采用原型模型 →
- 开发阶段采用迭代模型 →
- 交付阶段采用时间盒管理

2.4 项目管理过程（Project Management Process）

主要内容：

- 项目规划（Project Planning）
- 风险管理（Risk Management）
- 进度与成本控制（Scheduling & Cost Control）
- 质量保证（Quality Assurance）
- 人员与沟通管理（Team & Communication）

■ 核心目标：

在时间、成本与质量三者之间取得平衡。

2.5 小结（Summary）

| 主题   | 要点            |
|------|---------------|
| 软件过程 | 定义了软件开发的框架与步骤 |

| 主题   | 要点              |
|------|-----------------|
| 主要活动 | 需求、设计、实现、测试、维护  |
| 常见模型 | 瀑布、原型、迭代、RUP、敏捷 |
| 管理过程 | 规划、控制、质量、沟通     |

✅ 软件过程模型的选择取决于项目规模、团队结构与需求稳定性。



# 自测练习 (Self-Review)

## 一、选择题

1. 瀑布模型的主要缺点是 ( )。
- A. 过程太灵活    B. 需求变更困难    C. 缺少文档    D. 无法测试
2. 原型模型适用于 ( )。
- A. 需求稳定项目    B. 需求不明确项目    C. 长期维护项目    D. 小型项目
3. 敏捷开发的核心是 ( )。
- A. 文档驱动    B. 人员沟通与反馈    C. 阶段划分    D. 成本控制

## 二、判断题

1. ( ) RUP 是一种以文档为中心的线性模型。
2. ( ) XP 强调结对编程与持续反馈。
3. ( ) 时间盒模型要求在固定周期内完成交付。

## 三、简答题

1. 简述瀑布模型与迭代模型的主要区别。
2. 简要说明敏捷开发的核心价值观。
3. 软件过程与软件项目的关系是什么？



# 答案参考

选择题：

1. B
2. B
3. B

#### 判断题：

1. ×
2. √
3. √

#### 简答题要点：

1. 瀑布模型线性顺序，迭代模型循环渐进。
2. 强调个体与互动、可运行软件、客户合作、应对变化。
3. 软件过程定义方法论，软件项目在实际中实现该方法。

第二章结束 by 杜舟逸

## 第三章：软件需求分析和软件需求规格 (Software Requirements Analysis & Specification)

参考教材：Pankaj Jalote, 《软件工程导论》(A Concise Introduction to Software Engineering)

本章重点：理解软件需求分析的目标、过程和规格说明的编写方法。

### 学习目标

学习本章后，你应该能够：

- 理解什么是“好的软件需求规格”(SRS) 的特征
- 掌握需求分析的流程与活动
- 能编写软件需求规格说明书 (SRS)
- 熟悉用例驱动的功能建模方法
- 理解数据流图 (DFD) 和实体关系图 (ER图) 的分析作用

# 3.1 好软件需求规格的意义（Importance of Good SRS）

## 什么是 SRS？

- **定义：** 软件需求规格说明书（Software Requirements Specification, SRS）是用户需求与系统设计之间的**契约文档**。
- **作用：**
  - i. 明确系统“要做什么”，而非“怎么做”
  - ii. 为设计、测试和维护提供依据
  - iii. 降低沟通成本与需求歧义

## 一份好的 SRS 应具备：

| 特性   | 说明            |
|------|---------------|
| 完整性  | 所有功能和约束都被描述   |
| 一致性  | 各部分不冲突        |
| 可验证性 | 可通过测试确认需求是否满足 |
| 可追踪性 | 每个需求来源清晰可追溯   |
| 可修改性 | 结构清晰，易于更新     |

 **思考：** 在项目中，一个模糊的需求说明可能造成哪些问题？

# 3.2 需求过程（The Requirement Process）

## 主要阶段：

1. **需求获取（Elicitation）**
  - 与用户访谈、问卷、观察业务流程
  - 产出：初步需求列表
2. **需求分析（Analysis）**
  - 分类、去歧义、确定优先级
  - 输出：分析模型（DFD、用例）
3. **需求规格化（Specification）**
  - 编写标准化的 SRS 文档
4. **需求验证（Validation）**
  - 检查完整性、一致性和可行性
5. **需求管理（Management）**

- 控制需求变更，保持版本一致

### 3.3 需求规格（Software Requirement Specification）

#### 3.3.1 软件需求规格应具备的特点

- 正确性（Correctness）
- 无歧义性（Unambiguity）
- 完整性（Completeness）
- 一致性（Consistency）
- 可验证性（Verifiability）
- 可修改性（Modifiability）

#### 3.3.2 软件需求规格的组成

| 类型     | 内容              |
|--------|-----------------|
| 功能需求   | 系统应执行的操作        |
| 非功能需求  | 性能、可靠性、安全性、可维护性 |
| 外部接口需求 | 与硬件、外部系统交互方式    |
| 约束条件   | 法规、设计限制等        |

#### 3.3.3 需求文档的结构（SRS Template）

典型结构：

1. 引言（目的、范围、定义）
2. 总体描述（系统环境、用户特征）
3. 具体需求（功能、性能、接口）
4. 附录（数据字典、术语表）

### 3.4 用例驱动功能规格（Use Case–Driven Specification）

#### 3.4.1 基础知识

- **用例（Use Case）**：系统从用户角度描述的交互行为。
- **参与者（Actor）**：与系统交互的角色（人或系统）。

- 每个用例包含：
  - 触发事件
  - 正常流程
  - 备选流程
  - 异常情况

### 3.4.2 几个例子

例：图书馆管理系统

| 用例名称 | 借书                     |
|------|------------------------|
| 参与者  | 读者                     |
| 主要流程 | 输入图书编号 → 系统验证 → 更新借阅记录 |
| 异常流程 | 图书不可借、超期未还             |

### 3.4.3 扩展（Extensions）

- 使用 UML 用例图展示系统行为全貌
- 支持“扩展点”与“包含关系”以简化复杂功能

### 3.4.4 构建用例（Building Use Cases）

1. 识别系统边界
2. 确定主要参与者
3. 为每个参与者定义目标与场景
4. 用统一格式描述每个用例
5. 校验覆盖率与一致性

## 3.5 其他分析方法（Other Analysis Techniques）

### 3.5.1 数据流图（DFD）

- 描述系统中**数据如何流动**。
- 主要元素：
  - 过程（Process）
  - 数据流（Data Flow）
  - 数据存储（Data Store）
  - 外部实体（External Entity）

■ 层次化建模：

- 一级DFD：展示主要流程
- 二级DFD：细化某个子过程

3.5.2 实体关系图（ER图）

- 用于建模系统中的数据结构与关系。
- 组成部分：
  - 实体（Entity）
  - 属性（Attribute）
  - 关系（Relationship）

示例：

“学生” 实体与 “课程” 实体 之间存在 “选修” 关系。

3.6 验证（Validation）

- 目标：确保需求规格正确反映用户意图。
- 常用方法：
  - 需求评审（Review）
  - 原型验证（Prototype）
  - 模拟或场景分析（Simulation）

验证问题包括：

- 是否遗漏功能？
- 是否存在冲突？
- 是否具备可测性？

3.7 小结（Summary）

| 内容     | 关键点                     |
|--------|-------------------------|
| 需求过程   | 获取 → 分析 → 规格化 → 验证 → 管理 |
| SRS 特点 | 完整、一致、无歧义、可修改           |
| 功能建模   | 用例图、DFD                 |
| 数据建模   | ER 图                    |

| 内容 | 关键点          |
|----|--------------|
| 验证 | 保证需求的正确性与可行性 |

✅ 一份高质量的 SRS 是项目成功的基础。

## 🗨 自测练习 (Self-Review)

### 一、选择题

- 1. SRS 的主要作用是 ( )。  
A. 描述软件实现方法   B. 描述系统需求   C. 测试策略   D. 编码规范
- 2. 数据流图主要用于描述 ( )。  
A. 程序算法   B. 数据流动   C. 类关系   D. 界面布局
- 3. ER 图用于建模 ( )。  
A. 数据结构   B. 流程逻辑   C. 模块关系   D. 程序控制流

### 二、判断题

- 1. ( ) 需求分析阶段的输出是软件设计文档。
- 2. ( ) 功能需求描述系统要做什么。
- 3. ( ) 用例只适用于大型项目。

### 三、简答题

- 1. 请说明自顶向下与自底向上的需求分析差异。
- 2. 一份好的 SRS 具备哪些特征？
- 3. 用例图与DFD的主要区别是什么？

## 🧩 答案参考

选择题：

1. B   2. B   3. A

判断题：

1. × (输出是 SRS)   2. √   3. ×

简答题要点：



1. 自顶向下法从系统整体出发，自底向上法从细节组合。
2. 特征：完整、一致、无歧义、可追踪、可修改、可验证。
3. 用例图强调用户交互，DFD 强调数据流动。

第三章结束 by 侯博森

## 第四章：软件计划（Software Project Planning）

参考教材：Pankaj Jalote, 《软件工程导论》(A Concise Introduction to Software Engineering)

本章重点：软件项目的估算、计划、风险与监控

### 学习目标

通过学习本章，学生应当能够：

- 说明软件项目计划的意义
- 掌握自顶向下与自底向上的工作量估算方法
- 制定项目进度表与人员配备计划
- 理解质量计划与风险管理的主要步骤
- 能够设计项目监控与详细日程安排

## 4.1 工作量估算（Effort Estimation）

### 4.1.1 自顶向下估算方法（Top-Down Estimation）

- 从整体系统目标出发，估计整个项目所需的时间、人力与成本。
- 通常由项目经理或有经验的开发者完成。
- 优点：快速、整体性强；
- 缺点：对细节掌握不足，误差较大。

常用步骤：

1. 估计项目总规模（功能点或LOC）。
2. 根据以往经验确定平均生产率。
3. 计算人月和周期。

示例：若预计系统为10,000行代码，生产率为500行/人月，则约需20人月。

## 4.1.2 自底向上估算方法（Bottom-Up Estimation）

- 从各个模块或任务单元出发，逐个估算后求和。
- 优点：更精确，适合详细设计阶段。
- 缺点：需要完整设计信息，耗时长。

### 流程：

1. 将项目分解为较小任务（WBS）。
2. 对每个任务估算工作量与资源。
3. 汇总得出总成本与进度。

建议：在初期使用自顶向下法，设计阶段再用自底向上法修正。

## 4.2 项目进度和人员配备（Scheduling & Staffing）

- 根据估算结果，制定任务时间表并安排人员。

### 关键要素：

- 活动分解与依赖分析
- 里程碑（Milestones）
- 关键路径（Critical Path）
- 人力分布（Rayleigh 曲线）

### 人员配备策略：

- 项目初期人少、中期人多、后期再减少。
- 团队结构可为：
  - 职能型（按部门分）
  - 项目型（按模块分）
  - 矩阵型（双线管理）

## 4.3 质量计划（Quality Planning）

- 确保软件在交付时满足既定的质量标准。

### 主要目标：

- 明确质量标准与目标。

- 制定质量保证活动（评审、测试、度量）。
- 确定质量责任人和报告机制。

常用度量：

- 缺陷密度（Defect Density）
- 模块复杂度（Cyclomatic Complexity）
- 测试覆盖率（Test Coverage）

## 4.4 风险管理计划（Risk Management Plan）

### 4.4.1 风险管理的观念（Concept of Risk Management）

风险是“可能影响项目目标的潜在事件”。

风险管理目的：

- 识别潜在风险
- 分析其可能性与影响
- 制定预防与应对措施

### 4.4.2 风险评估（Risk Assessment）

步骤：

1. 识别风险（技术、人员、资源、需求、外部）
2. 分析风险：确定概率和影响
3. 量化风险暴露：

$$\text{Risk Exposure} = \text{Probability} \times \text{Impact}$$

示例：

| 风险     | 概率  | 影响 | 暴露量   |
|--------|-----|----|-------|
| 关键成员离职 | 0.3 | 高  | 0.3×高 |

### 4.4.3 风险控制（Risk Control）

策略：

- **规避 (Avoid)**: 通过改变计划消除风险。
- **转移 (Transfer)**: 外包或保险。
- **缓解 (Mitigate)**: 降低概率或影响。
- **接受 (Accept)**: 准备应急方案。

#### 4.4.4 一个实用的风险管理计划方法 (A Practical Approach)

1. 识别风险清单。
2. 设定风险等级与优先级。
3. 为每个高风险制定应对计划。
4. 定期更新风险状态表。
5. 建立风险责任制。

### 4.5 项目监测计划 (Project Monitoring Plan)

#### 4.5.1 项目的度量 (Project Metrics)

- **进度偏差 (SV)** = 实际完成进度 - 计划进度
- **成本偏差 (CV)** = 实际成本 - 预算成本
- **质量度量**: 缺陷数、修复率

#### 4.5.2 项目监测和跟踪 (Monitoring & Tracking)

方法:

- 定期状态报告
- 里程碑评审会议
- 变更控制流程 (Change Control)
- 项目仪表盘 (Dashboard)

目标:

- 提前发现问题
- 调整资源与进度
- 保证项目按时高质量完成

# 4.6 详细日程安排（Detailed Scheduling）

- 将任务细分，形成详细时间表。
- 使用工具：
  - 甘特图（Gantt Chart）
  - 网络图 / PERT 图
  - 任务依赖矩阵

## 关键路径分析（Critical Path Analysis）

关键路径上的任务延迟将导致整个项目延迟。  
应优先监控关键路径任务。

# 4.7 小结（Summary）

| 主题 | 关键内容                       |
|----|----------------------------|
| 估算 | 自顶向下 / 自底向上；LOC、功能点、COCOMO |
| 进度 | 关键路径、里程碑、Rayleigh 曲线       |
| 质量 | 预防为主，测试与度量                 |
| 风险 | 识别 → 评估 → 控制               |
| 监控 | 指标化管理、定期跟踪                 |
| 日程 | 甘特图、PERT分析                 |

✅ 一个好的项目计划，应当是动态的、可跟踪的、可度量的。

## 🧠 自测练习（Self-Review Questions）

### 一、选择题

1. 自顶向下估算的优点是（ ）。  
A. 精确    B. 快速    C. 可控    D. 复杂
2. 以下哪项不属于风险控制策略？（ ）  
A. 缓解    B. 规避    C. 隐藏    D. 接受
3. Rayleigh 曲线描述的是（ ）  
A. 成本变化    B. 人力随时间的分布    C. 缺陷趋势    D. 代码规模

二、判断题

- 1. ( ) 风险评估只需在项目初期进行一次。
- 2. ( ) 甘特图可用来表示任务之间的依赖关系。
- 3. ( ) 自底向上估算比自顶向下估算更适合需求分析阶段。

三、简答题

- 1. 比较自顶向下与自底向上估算的优缺点。
- 2. 简述一个有效的风险管理计划的主要步骤。
- 3. 如何利用度量结果改进项目监控？

 答案参考

选择题：

1. B    2. C    3. B

判断题：

- 1. × （风险需动态更新）
- 2. √
- 3. × （自底向上适合详细设计阶段）

简答题参考要点：

1.

| 方法   | 优点        | 缺点        |
|------|-----------|-----------|
| 自顶向下 | 快速、便于早期规划 | 误差大、依赖经验  |
| 自底向上 | 精确、基于细节   | 耗时长、需完整设计 |

- 2. 风险管理步骤：识别 → 分析 → 控制 → 监控。
- 3. 通过监测度量指标（进度偏差、缺陷率等），可发现问题、调整资源并持续优化过程。

第四章结束 by 董博明

# 第五章：软件体系结构（Software Architecture）

参考教材：Pankaj Jalote, 《软件工程导论》（A Concise Introduction to Software Engineering）

本章重点：理解软件体系结构的作用、常见结构视图、构件与连接件，以及体系结构模式。

## 学习目标

通过学习本章，你应能：

- 理解软件体系结构的概念与作用
- 掌握体系结构视图的类型及其表达方式
- 理解“构件（Component）”与“连接件（Connector）”的含义
- 熟悉典型体系结构模式（如管道-过滤器、共享数据、客户机-服务器等）
- 了解体系结构设计的文档化与评估方法

## 5.1 软件体系结构的作用（Role of Software Architecture）

### 概念

- 软件体系结构（Software Architecture）是**系统的高层结构设计**，定义了：
  - 系统的主要组件（Components）
  - 各组件间的交互方式（Connectors）
  - 整体组织与约束（Configuration）

### 作用

1. **指导系统设计与实现**  
体系结构是设计阶段的蓝图。
2. **支持沟通**  
便于开发者、测试者、客户理解系统结构。
3. **影响非功能属性**  
性能、可扩展性、安全性等往往由体系结构决定。
4. **促进复用**  
可通过模式与框架复用结构。

■ 例子：

浏览器的体系结构包括渲染引擎、脚本引擎、网络模块、UI 层等，彼此通过定义好的接口通信。

5.2 体系结构视图（Architectural Views）

为什么需要多视图

- 软件系统复杂，一个单一视角难以全面描述。
- 采用多个“视图”(View) 来从不同角度展示系统结构。

常见视图类型

| 视图                                  | 描述           |
|-------------------------------------|--------------|
| 模块视图（Module View）                   | 展示代码模块及依赖    |
| 组件-连接视图（Component & Connector View） | 展示运行时组件与交互   |
| 分配视图（Allocation View）               | 展示软件与硬件的映射关系 |

🔍 实际中常采用“4+1视图模型”：逻辑、开发、过程、物理 + 用例视图。

5.3 构件和连接件视图（Component & Connector View）

5.3.1 构件（Components）

- 定义：系统的基本运行单元，提供一组功能。
- 类型：
  - 数据构件（Data Component）
  - 功能构件（Functional Component）
  - 控制构件（Control Component）

✚ 示例：

在一个电子商务系统中：

- “订单管理” 是一个功能构件
- “数据库接口” 是数据构件
- “请求调度器” 是控制构件



## 5.3.2 连接件（Connectors）

- 定义：连接构件之间通信的机制。
- 类型：
  - 调用（Procedure Call）
  - 事件（Event）
  - 消息队列（Message Queue）
  - 数据流（Data Flow）
- 连接件决定了系统中组件的交互风格。

## 5.3.3 举例（Example）

客户端-服务器系统：

- 构件：客户端、服务器
- 连接件：网络通信（Socket、HTTP）
- 体系结构描述：  
客户端通过请求-响应机制访问服务器资源。

## 5.4 构件和连接件视图的体系结构模式（Architectural Styles）

体系结构模式是一种**可复用的结构组织方式**。

不同模式适合不同类型的系统。

### 5.4.1 管道-过滤器模式（Pipe-and-Filter）

- 结构：多个“过滤器”通过“管道”顺序连接。
- 特点：
  - 每个过滤器独立处理输入数据并输出结果；
  - 易扩展、易并行。
- 应用：编译器、数据流处理。

■ 例子：

编译器体系结构

源代码 → 词法分析 → 语法分析 → 语义分析 → 代码生成

### 5.4.2 共享数据模式（Shared Data Style）

- 各组件通过**共享数据存储**（如数据库、文件系统）进行通信。
- 特点：
  - 数据集中存放；
  - 组件间耦合低；
  - 容易维护，但安全与同步需注意。
- 应用：银行系统、信息管理系统。

### 5.4.3 客户端-服务器模式（Client-Server Style）

- 服务器提供服务，客户端发起请求。
- 特点：
  - 支持分布式；
  - 扩展性好；
  - 需考虑通信延迟与安全性。
- 应用：Web应用、邮件系统。

### 5.4.4 其他模式（Other Styles）

| 模式                  | 特点            | 应用场景        |
|---------------------|---------------|-------------|
| 分层模式（Layered Style） | 系统分为多层，层间顺序通信 | 操作系统、网络协议栈  |
| 事件驱动模式（Event-Based） | 松耦合、异步通信      | 前端系统、实时响应系统 |
| 仓库模式（Repository）    | 数据集中存储        | 数据分析系统      |

## 5.5 体系结构设计的文档化（Architecture Documentation）

目的：

- 清晰表达体系结构设计决策
- 方便团队沟通与后续维护

常用文档内容：

- 1. 构件和连接件描述
- 2. 模式选择与理由
- 3. 视图图示（组件图、部署图）
- 4. 设计约束与假设

常用工具： UML、ArchiMate、PlantUML

## 5.6 体系结构评估（Architecture Evaluation）

### 为什么要评估？

- 体系结构设计直接影响系统质量。
- 早期发现潜在问题可降低成本。

### 常用方法

| 方法                                          | 说明          |
|---------------------------------------------|-------------|
| ATAM（Architecture Tradeoff Analysis Method） | 分析不同质量属性的权衡 |
| SAAM（Software Architecture Analysis Method） | 基于场景的结构评估   |
| 评审与专家检查                                     | 小型项目中常用     |

### 评估维度：

- 性能（Performance）
- 可扩展性（Scalability）
- 可维护性（Maintainability）
- 安全性（Security）

## 5.7 小结（Summary）

| 内容     | 关键点                |
|--------|--------------------|
| 体系结构   | 系统的高层设计蓝图          |
| 构件与连接件 | 功能单元 + 通信机制        |
| 常见模式   | 管道-过滤器、共享数据、C/S、分层 |

| 内容  | 关键点        |
|-----|------------|
| 文档化 | 表达结构与决策    |
| 评估  | 保证结构质量与合理性 |

✅ 体系结构是软件设计的“骨架”，决定了系统的灵活性与可扩展性。

## 自测练习 (Self-Review)

### 一、选择题

- 1. 管道-过滤器模式的典型特征是 ( )。  
A. 组件并行通信    B. 数据集中存储    C. 请求-响应结构    D. 层次结构
- 2. 客户端-服务器体系结构的主要优势是 ( )。  
A. 高耦合    B. 易扩展    C. 低性能    D. 无安全性
- 3. 构件与连接件对应于 ( )。  
A. 数据与控制流    B. 模块与通信机制    C. 接口与类    D. 文档与测试

### 二、判断题

- 1. ( ) 构件是系统的运行单元，连接件是交互机制。
- 2. ( ) 管道-过滤器模式不适合并行处理。
- 3. ( ) 分层模式能增强系统的可维护性。

### 三、简答题

- 1. 比较“管道-过滤器模式”和“共享数据模式”的异同。
- 2. 什么是体系结构文档？应包含哪些主要内容？
- 3. 简述 ATAM 方法的主要思想。

## 答案参考

选择题：

1. A    2. B    3. B

判断题：

1. √    2. ×    3. √

简答题要点：

1.

| 模式     | 通信方式  | 优点      | 缺点      |
|--------|-------|---------|---------|
| 管道-过滤器 | 顺序数据流 | 易扩展、易并行 | 交互受限    |
| 共享数据   | 数据库共享 | 解耦、集中管理 | 安全与同步复杂 |

2. 体系结构文档用于描述系统的结构、模式选择、设计决策、约束及视图图示。
3. ATAM 通过评估体系结构在多种质量属性间的权衡（如性能与可扩展性），帮助发现潜在风险。

第五章结束 by 高铭希

# 第六章：设计（Software Design）

第六章学习指南：设计（Software Design）

## 6.1 设计的基本概念

核心思想

设计是将需求分析结果转化为软件系统的结构、模块、接口和数据等细节的过程。

关键概念

• 耦合 (Coupling)

模块之间的依赖程度。

- 高耦合：模块相互依赖强 → 难以维护。
- 低耦合：模块相互独立 → 易于复用。
- 💡 前端游戏示例：将游戏逻辑（JS）与界面样式（CSS）解耦，保持低耦合。

• 内聚 (Cohesion)

模块内部元素之间的关联程度。

- 高内聚：模块只负责单一功能。
- 低内聚：模块功能混乱。
- 💡 示例：scoreManager.js 只处理分数，不负责渲染。

• 开闭原则 (Open-Closed Principle)

软件应对扩展开放，对修改关闭。

- 💡 游戏例子：新增敌人类型时，可以继承 Enemy 类，而不必修改原有代码。

## 6.2 面向功能设计

### 💡 概念

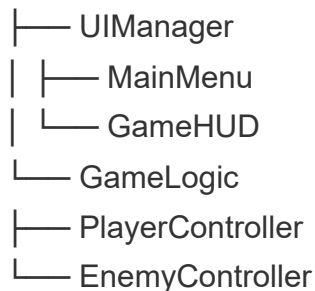
以系统的功能为中心，将系统划分为若干个功能模块。

### 📊 结构图 (Structure Chart)

用来展示系统的模块层次结构及其调用关系。

### 💡 示例：

GameSystem



### 🔧 结构化设计方法

- 自顶向下分解 (Top-down Design)  
先设计总体，再逐步细化子模块。
- 模块化设计 (Modular Design)  
每个模块只完成一类功能，接口清晰。

### 💡 游戏开发思路应用

HTML + JS 游戏中，可以将逻辑拆解为：

- Game.js：控制主循环
- Player.js：玩家行为模块
- Enemy.js：敌人模块
- UI.js：渲染与交互模块

## 6.3 面向对象设计 (OOD)

### 💡 基本概念

以对象为核心，每个对象包含属性（数据）和方法（行为）。

### 💡 示例 (JS)：

```

class Player {
  constructor(name, health) {
    this.name = name;
    this.health = health;
  }
  attack(target) {
  
```

```
target.health -= 10;
}
}
```

### ◆ UML（统一建模语言）

- 类图（Class Diagram）：显示对象之间的关系。
- 顺序图（Sequence Diagram）：描述对象之间的交互顺序。
- 状态图（State Diagram）：表示对象状态变化。

#### 💡 游戏例子：

UML 类图可以描述 Player、Enemy、Weapon 的继承关系。

### 💬 设计方法论

- 抽象：提取共同特征。
- 封装：隐藏内部实现。
- 继承：重用已有代码。
- 多态：相同接口，不同实现。

---

## 6.4 详细设计

### ⚙️ 逻辑/算法设计

在实现之前，确定具体算法与控制逻辑。

#### 💡 示例：

玩家移动算法或敌人 AI 路径规划。

### 🌱 类状态模型

描述对象在不同事件下的状态变化。

#### 💡 示例：

Player 状态机：

Idle → Running → Jumping → Attacking → Idle

---

## 6.5 验证（Verification）

确认设计是否满足需求规格说明。

- 检查模块接口的一致性。
  - 确保 UML、结构图与需求一致。
- 💡 游戏例：验证 UIManager 是否正确响应游戏状态变化。

---

## 6.6 复杂性度量

度量设计的复杂程度，帮助改进可维护性。

### 6.6.1 面向功能设计复杂性度量

例如：模块间调用路径数、接口数量。

### 6.6.2 面向对象设计复杂性度量

如类的继承层次、方法数量、对象交互数量。

💡 示例：

- 若 GameManager 管理太多对象（高复杂度），应拆分模块。

---

## 6.7 小结

软件设计的目标：

- 提高可维护性、可扩展性与复用性
- 降低耦合度，提高内聚度
- 确保设计能平稳过渡到编码阶段

💡 前端游戏思考：

设计良好的架构，可以让你轻松扩展关卡、角色、UI，不会因为新增一个功能就改动全局。

第六章结束 by 云建皓

# 第7章 – 编码与单元测试

## 7.1 编码的重要性

- 编码是将设计转化为可执行程序的阶段。
- 高质量的编码直接影响：
  - 软件的可靠性与可维护性。
  - 测试与调试的效率。
- 编码阶段约占整个开发成本的 20%~30%。

## 7.2 编码的目标

### 1. 正确性 (Correctness)

程序必须实现设计和规格说明所要求的功能()。



## 2. 可读性 (Readability)

代码应易于他人理解和维护,减少出现需要注释解释的情况。

## 3. 可维护性 (Maintainability)

修改和扩展应方便、安全。

## 4. 效率 (Efficiency)

保证在合理资源下运行良好。

# 7.3 编码风格与规范

- 统一的编码风格有助于：
  - 提高可读性和可维护性。
  - 便于团队协作和代码审查。
- 常见的编码规范包括：
  - 命名规则（变量、函数、类命名清晰且一致）。
  - 注释标准（解释复杂逻辑、记录作者和修改信息）。
  - 缩进与排版（保持一致结构）。
  - 模块结构清晰，每个函数只完成一个逻辑任务。
- **代码审查 (Code Review)** 是确保编码质量的重要环节。

# 7.4 防御式编程 (Defensive Programming)

- 编程时要假设可能出现错误输入或异常情况。
- 常用做法：
  - 输入检查与验证。
  - 使用断言 (assertions) 检测程序状态。
  - 错误处理与异常机制。
  - 限制副作用和全局变量使用。
- 目标：让程序在异常情况下仍能保持可预测行为。

# 7.5 代码审查与走查 (Code Review and Walkthrough)

- **代码审查 (Review)：**
  - 由其他开发人员检查代码。
  - 关注逻辑错误、风格问题、潜在缺陷。
- **走查 (Walkthrough)：**
  - 作者带领团队逐步解释代码实现。

- 强调理解与学习，而非批评。
- 优点：
  - 早期发现错误。
  - 提高团队知识共享。
  - 降低测试阶段的错误修复成本。

## 7.6 单元测试 (Unit Testing)

- **定义：** 测试软件的最小可测试单元（通常是一个函数或类）。
- **目的：** 验证单元功能正确性与逻辑完整性。
- **特点：**
  - 通常由开发人员执行。
  - 测试范围小、速度快。
- **常见内容：**
  - 输入与输出验证。
  - 边界条件检查。
  - 异常与错误处理。
- **工具：** 常用如 JUnit、CppUnit、pytest 等。

## 7.7 编码与单元测试的关系

- 编码与单元测试应**同时规划、交叉进行**。
- 良好的编码风格能简化测试。
- 测试结果可反过来揭示代码设计中的问题。
- 在敏捷开发中，测试驱动开发（TDD）体现了两者的结合：
  - **先写测试，再写代码。**

## 7.8 编码过程中的质量保证

- 编码阶段是质量控制的第一道关口。
- 关键措施：
  - i. 遵循标准与最佳实践。
  - ii. 定期进行代码审查。
  - iii. 使用静态分析工具（如检查未使用变量、潜在缺陷）。
  - iv. 进行充分的单元测试。

## 7.9 常见编码错误类型

- 逻辑错误（逻辑条件不符）。
- 边界错误（循环范围、数组越界）。
- 初始化错误（变量未初始化）。
- 资源泄漏（内存、文件未释放）。
- 并发错误（多线程共享资源冲突）。

## 7.10 小结

- 编码阶段不仅仅是写代码，而是**构建高质量软件的核心环节**。
- 好的代码应具备：
  - 清晰、规范、可维护。
  - 具备防御性，能应对异常。
  - 经过审查与充分的单元测试。
- “**写出正确的代码**”与“**证明代码正确**”同样重要。

## 关键术语

| 术语                            | 含义              |
|-------------------------------|-----------------|
| Coding                        | 将设计转化为可执行程序的过程  |
| Coding Standards              | 编码规范与风格约定       |
| Defensive Programming         | 防御式编程，提高容错能力    |
| Code Review                   | 他人检查代码质量        |
| Walkthrough                   | 作者带领团队理解代码      |
| Unit Testing                  | 测试最小的软件组件       |
| TDD (Test-Driven Development) | 测试驱动开发，先写测试再写代码 |

第七章结束 by 官俊屹

# 第八章：集成与系统测试（Integration and System Testing）

教材参考：Pankaj Jalote, 《软件工程导论》（A Concise Introduction to Software Engineering）

本章目标：理解从单元测试到系统测试的各个阶段，掌握集成策略、测试方法与系统验收标准。

## 学习目标（Learning Objectives）

学习完本章后，你应该能够：

- 理解软件测试的层次结构与目标
- 掌握集成测试的主要策略（自顶向下、自底向上、混合）
- 理解系统测试的主要类型与流程
- 熟悉回归测试与验收测试的概念
- 了解自动化测试与测试计划的重要性

## 8.1 软件测试概述（Overview of Software Testing）

定义：

软件测试是验证系统是否满足需求并发现潜在缺陷的过程。

主要目标：

1. 发现错误（Error Detection）
2. 验证需求（Requirement Verification）
3. 评估质量（Quality Assessment）

 测试层次：

- 单元测试 → 集成测试 → 系统测试 → 验收测试  
(Unit) (Integration) (System) (Acceptance)

## 8.2 集成测试（Integration Testing）

在单元测试完成后，对模块之间的交互进行测试，以发现接口错误和逻辑缺陷。

## 8.2.1 目标 (Objectives)

- 验证模块间接口正确性
- 检查数据流和控制流
- 评估系统功能组合后的行为

## 8.2.2 集成策略 (Integration Strategies)

### 1 自顶向下集成 (Top-Down Integration)

特点：

- 从系统的顶层模块开始，逐步集成下层模块。
- 未完成的模块由“桩模块 (Stubs)”替代。

优点：

- 早期验证系统架构
- 缺陷定位容易

缺点：

- 需编写大量桩模块
- 底层功能测试较晚

示意图：

- Main Module
  - ├ Stub A
  - ├ Stub B
  - └ Stub C

### 2 自底向上集成 (Bottom-Up Integration)

特点：

- 从底层模块开始测试，再逐步向上集成。
- 使用“驱动模块 (Drivers)”模拟上层调用。

优点：

- 底层逻辑验证早
- 不需要桩模块

**缺点：**

- 系统功能验证较晚

**示意图：**

- Lowest Level Modules → Combine into Subsystems → Full System

**3 混合集成（Sandwich / Hybrid Integration）**

结合前两种策略：

- 上层模块采用自顶向下集成
- 下层模块采用自底向上集成
- 中间层同时进行合并测试

**优点：**

- 灵活高效
- 适合中大型项目

**8.2.3 增量集成（Incremental Integration）**

每次仅加入一个或几个模块进行测试，逐步扩展系统范围。

相比“一次性集成（Big-Bang Integration）”，风险更小、定位更容易。

**8.3 系统测试（System Testing）****定义：**

对整个集成系统进行全面测试，验证其功能、性能、安全性及兼容性。

**8.3.1 功能测试（Functional Testing）**

验证软件是否满足功能需求说明（SRS）。

常见方法：

- 黑盒测试（Black Box Testing）
- 等价类划分（Equivalence Partitioning）
- 边界值分析（Boundary Value Analysis）

## 8.3.2 性能测试 (Performance Testing)

测试系统在不同负载下的响应时间与吞吐量。

包括：

- 压力测试 (Stress Testing)
- 负载测试 (Load Testing)
- 稳定性测试 (Stability Testing)

## 8.3.3 安全性与可靠性测试 (Security & Reliability Testing)

**安全测试：**

防止未经授权的访问与数据泄露。

**可靠性测试：**

验证系统在长期运行下的稳定性与容错能力。

## 8.3.4 可用性与兼容性测试 (Usability & Compatibility Testing)

- **可用性测试：**评估界面友好度与用户体验。
- **兼容性测试：**验证系统在不同平台、浏览器、硬件下能否正常运行。

## 8.3.5 回归测试 (Regression Testing)

在修改或修复错误后，重新执行测试，确保旧功能未受影响。

 常见策略：

- 自动化测试脚本 (Automated Regression Scripts)
- 测试用例版本控制 (Versioned Test Cases)

## 8.3.6 验收测试 (Acceptance Testing)

由客户或最终用户执行，用于确认系统是否可交付。

**类型：**

| 类型       | 说明        |
|----------|-----------|
| Alpha 测试 | 内部环境下用户参与 |
| Beta 测试  | 真实用户环境下测试 |

## 8.4 测试自动化与管理（Test Automation and Management）

### 8.4.1 自动化测试工具（Automation Tools）

常见工具：

| 工具         | 语言/平台  | 功能       |
|------------|--------|----------|
| Selenium   | Web    | UI 自动化测试 |
| JUnit      | Java   | 单元与集成测试  |
| pytest     | Python | 自动化测试框架  |
| Postman    | API    | 接口测试     |
| LoadRunner | 多语言    | 性能测试     |

### 8.4.2 测试计划与报告（Test Planning & Reporting）

一个完善的测试计划应包含：

- 测试目标（Objectives）
- 测试范围与优先级（Scope & Priority）
- 测试环境与工具（Environment & Tools）
- 测试用例与执行计划（Test Cases & Schedule）
- 缺陷跟踪与报告机制（Defect Tracking）



# 8.5 小结 (Summary)

| 主题   | 要点                |
|------|-------------------|
| 测试层次 | 单元 → 集成 → 系统 → 验收 |
| 集成策略 | 自顶向下、自底向上、混合、增量   |
| 系统测试 | 功能、性能、安全、兼容性      |
| 测试管理 | 自动化与报告体系          |

✅ 测试的目标是发现错误，而不是证明程序“没有错误”。

## 🧠 自测练习 (Self-Review)

### 一、选择题

1. 自顶向下集成测试需要使用 ( )。
- A. 驱动模块   B. 桩模块   C. 模拟器   D. 负载器
2. 回归测试的主要目的是 ( )。
- A. 提高性能   B. 验证修复后的稳定性   C. 测试安全性   D. 增强兼容性
3. Alpha 测试通常由 ( )。
- A. 开发团队   B. 用户在真实环境   C. 内部用户参与   D. 客户代表执行

### 二、判断题

1. ( ) 集成测试的目标是验证系统整体性能。
2. ( ) 回归测试在每次修改代码后都应执行。
3. ( ) 系统测试只关注功能正确性。

### 三、简答题

1. 简述自顶向下与自底向上集成测试的主要区别。
2. 系统测试包括哪些主要类型？
3. 解释什么是回归测试及其重要性。

 **答案参考****选择题：**

1. B
2. B
3. C

**判断题：**

1. ×
2. √
3. ×

**简答题要点：**

1. 自顶向下以桩模块替代下层；自底向上以驱动模块模拟上层。
2. 功能测试、性能测试、安全性测试、兼容性测试。
3. 回归测试是修改后验证系统未被破坏的重要手段，可通过自动化执行。

第八章结束 by 杜舟逸

study guide完